Projet – Une application multiplateforme

React Native · En trinôme · Du 27 octobre au 17 décembre

En trinôme, vous concevez une application React Native **sur le sujet de votre choix**. Elle fonctionne avec le même code dans un navigateur de bureau et sur un téléphone. Le projet avance à chaque TP : chaque TP est un **jalon** du projet final.

Le projet en bref

- **Groupes de trois**, formés le premier jour. Un binôme est possible si l'effectif l'impose, jamais une personne seule.
- **Sujet libre**, validé par moi avant le TP2.
- Il constitue la **note pratique** du module : **50 %** de la note. **Chaque membre est propriétaire d'une partie** : la note du groupe est modulée selon la sienne.
- Rendu : dernier commit poussé **le mercredi 16 décembre à 23h59**. Démo le 17 décembre.

1 Ce que l'application doit contenir

Obligatoire

- **Au moins 4 écrans**, organisés avec Expo Router : des onglets, une pile et au moins une route dynamique (par exemple `/items/[id]`).
- Sur le web, **l'URL suit la navigation** : une page de détail s'ouvre directement depuis son adresse.
- Un **catalogue responsive** : une grille sur ordinateur, une liste fluide sur téléphone, avec des **filtres** dont la logique vit dans un hook.
- Une **API publique** consommée avec `fetch()`, avec les états de chargement et d'erreur.
- **Firebase** : authentification par e-mail et **deux CRUD** (créer, lire, modifier, supprimer) sur deux collections différentes.
- Une **architecture MVC** en TypeScript : modèles typés, services, hooks contrôleurs, composants d'affichage purs.
- Une interface **lisible et cohérente** : hiérarchie visuelle, contrastes suffisants, retours visuels au toucher.
- Un `README.md` complet (voir la partie 4).

> Une application qui ne se lance pas sur le web **ou** sur Expo Go ne peut pas être évaluée. Testez les deux à chaque commit important.

Valorisé (bonus)

- Une API native du téléphone : caméra, localisation, notifications, retour haptique…
- Des animations soignées.
- Un cache ou un mode hors ligne.
- Des tests automatisés des fonctions du modèle.
- La version web déployée en ligne.
- Une bibliothèque d'état (Zustand, TanStack Query…), si vous savez expliquer ce choix.

2 Un propriétaire par partie

Chaque membre est **propriétaire** d'une partie de l'application. Il la conçoit, l'écrit et en répond au point d'étape comme à la démo. Les deux autres l'aident et relisent son code.

Partie	Contenu	Au fil des TP
A · Catalogue	Modèle et service de l'API publique, catalogue responsive, filtres	TP2 : modèle, service factice et catalogue. TP3 : API publique et filtres
B · Navigation et compte	Structure de navigation, écran de détail, authentification, premier CRUD	TP2 : modèle et service de la première collection. TP3 : navigation et détail. TP4 : authentification et premier CRUD
C · Données partagées	État partagé (Context), écran groupé (`SectionList`), rafraîchissement au focus, second CRUD	TP2 : Context et deuxième écran. TP3 : `SectionList` et cycle de vie. TP4 : second CRUD

> Indiquez le propriétaire de chaque partie dans le README **dès le TP2**. En binôme, répartissez les trois parties entre vous deux et indiquez-le de la même façon. Chacun doit aussi comprendre le reste de l'application : je peux poser une question sur une autre partie.

3 Des idées de sujets

Ces quatre sujets s'appuient sur des API **gratuites et sans clé**. Vous pouvez en proposer un autre : il doit se prêter à un catalogue et à deux CRUD.

Sujet	API publique	Idées de CRUD Firebase
Carnet de recettes	TheMealDB	Mes recettes personnelles, ma liste de courses
Bibliothèque personnelle	Open Library	Mes lectures (lu, en cours, à lire), mes avis
Carnet de voyage	REST Countries et Open-Meteo	Mes voyages, les étapes d'un voyage
Garde-manger anti-gaspi	Open Food Facts	Mes produits et leurs dates de péremption, mes listes de courses

Deux groupes d'une même classe ne peuvent pas choisir le même sujet. Le premier qui me l'envoie le garde.

4 Les jalons

Quand	Jalon	Ce qui doit exister
Mar. 27 oct.	**Lancement**	Trinôme formé, sujet proposé, dépôt commun créé, Chris-CHEVALIER invité
Mer. 28 oct.	**TP2**	Arborescence MVC, modèles typés, service de données factices, hook contrôleur, catalogue responsive, un Context
Du 29 oct. au 24 nov.	**TP3 (noté)**	Navigation complète (onglets, pile, détail), API publique, filtres, cycle de vie. Tag `jalon-2` au plus tard le 24 nov. à 23h59
Mer. 25 nov.	**TP4**	Authentification Firebase et deux CRUD
Mer. 16 déc. 23h59	**Rendu final**	Application complète, README, bonus éventuels
Jeu. 17 déc.	**Démo**	8 minutes par groupe : 5 min de démo sur le web et sur mobile, puis 3 min de questions

Le 25 novembre, pendant le TP4, je fais un **point d'étape** avec chaque groupe : 5 minutes de démo du jalon 2, où chacun montre sa partie, et une question à chaque membre. Ce jalon est **noté** (voir l'énoncé du TP3).

5 Le dépôt et le rendu

- Un dépôt GitHub **privé** nommé `rn-projet-nom1-nom2-nom3`, avec **Chris-CHEVALIER** invité dès le premier jour.
- **Chacun commite avec son propre compte**, régulièrement. L'historique me sert à mesurer la contribution de chacun.
- Le projet se lance avec `npm install` puis `npx expo start`, sans autre manipulation. Aucun fichier APK ou IPA n'est demandé.

Le README contient

- le sujet, les membres du groupe et le **propriétaire de chaque partie** ;
- les fonctionnalités, avec des captures d'écran sur le web et sur mobile ;
- l'API publique utilisée et le schéma des collections Firebase ;
- le parcours de navigation (la liste des écrans et leurs URL) ;
- les instructions de lancement, dont la configuration Firebase à fournir ;
- les bonus réalisés ;
- la déclaration d'usage de l'IA.

> **Rendu final**
>
> Le 16 décembre au plus tard, envoyez à chevalier@chris-freelance.com un e-mail avec l'objet `[RN-IWB1] Projet — NOM1, NOM2, NOM3` (ou `IWB2`) et le lien du dépôt. Je corrige le dernier commit poussé avant 23h59.

6 L'évaluation

Critère	Ce que je regarde	Points
Fonctionnalités	Les exigences obligatoires sont présentes et fonctionnent sur le web et sur mobile	5
Jalon 2 (TP3)	La note du TP3 sur 20, ramenée sur 3	3
Architecture	Séparation MVC, types, hooks, composants purs, un fichier par domaine	3
Interface	Responsive, lisible, cohérente, états de chargement, d'erreur et de liste vide	3
Qualité	Code lisible, `npx tsc --noEmit` sans erreur, commits réguliers, README complet	2
Démo et questions	Démo fluide, chaque membre présente sa partie et répond juste à une question sur son code	4
Bonus (dans la limite de 20)		+2

La note est celle du groupe, **modulée individuellement** d'après la partie dont chacun est propriétaire : son état, ses commits et ses réponses. Un membre qui n'a pas contribué peut avoir 0.

7 L'IA générative

- L'IA (ChatGPT, Claude, Copilot…) est **autorisée** pour le projet.
- Chaque membre doit **comprendre et savoir expliquer** chaque partie du code. Une fonctionnalité que personne ne sait expliquer pendant la démo n'est pas comptée.
- Déclarez votre usage dans le README : quels outils, pour quelles tâches.
- Méfiez-vous des réponses qui datent : `expo eject`, `expo-cli` global ou React Navigation configuré à la main ne correspondent pas à la version d'Expo utilisée dans ce module.